

NEXUS AUTOMATIONS PH

The Autonomous AI Agent & Facebook Page Playbook

Architecting Zero-Cost, Self-Sustaining Social Ecosystems

Elite Edition | Production-Ready Blueprints

100% Free-Tier Architecture

1. Executive Summary & Architecture Philosophy

The era of manual social media management is obsolete. Modern businesses require 24/7 responsiveness, hyper-personalized interactions, and consistent, high-quality content generation. This playbook provides the definitive architectural blueprint for transitioning from traditional community management to a **100% autonomous AI agent ecosystem**.

This master-class guide is engineered for **Nexus Automations PH** and its clients. It utilizes a meticulously selected stack of enterprise-grade tools that offer **permanent free tiers**. By leveraging serverless functions, robust cloud databases, and cutting-edge LLMs (Large Language Models), we eliminate overhead while maximizing scalability.

The Zero-Cost Paradigm: Every tool, database, and API referenced in this playbook requires zero monthly subscription fees, relying entirely on generous developer free-tiers capable of handling small to medium business loads.

2. Complete Free Tech Stack Blueprint

To achieve high availability without incurring costs, we distribute the workload across specialized serverless platforms.

Component	Tool / Platform	Free Tier Limits	Strategic Advantage
AI Engine	Google Gemini 1.5 Flash API	15 RPM, 1 million tokens/min	Lightning-fast inference, high context window, completely free for standard automation loads.
Database & Auth	Supabase (PostgreSQL)	500MB database, 2GB bandwidth	Production-ready Postgres with instant REST APIs and robust Row Level Security (RLS).
Orchestration	GitHub Actions	2,000 CI/CD minutes/month	Perfect for chron-based tasks (daily content generation/posting) without hosting n8n.
Webhook Host	Render / Vercel (Node.js)	500 hours/month / 100GB bandwidth	Serverless edge execution for instantly receiving and parsing Meta Webhooks.

Component	Tool / Platform	Free Tier Limits	Strategic Advantage
Social Graph	Meta Graph API	Standard rate limits (200 calls/hr/user)	Direct pipeline into Messenger and Facebook Page feeds.

3. Database & Security Implementation (Supabase)

Data integrity and security are paramount. We utilize Supabase (PostgreSQL) to store captured leads and interaction logs. The following SQL scripts define the schema and the exact **Row Level Security (RLS)** policies required to allow our webhooks to insert data securely while preventing unauthorized public reads.

Schema Creation

```
-- Enable UUID extension
CREATE EXTENSION IF NOT EXISTS "uuid-oss";

-- 1. Leads Table: Stores extracted user profiles and intents
CREATE TABLE public.leads (
  id UUID DEFAULT uuid_generate_v4() PRIMARY KEY,
  psid VARCHAR(255) UNIQUE NOT NULL, -- Page Scoped ID
  full_name VARCHAR(255),
  inferred_intent VARCHAR(100),
  lead_score INT DEFAULT 0,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  last_interaction TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- 2. Interaction Logs: Audits the AI's conversation history
CREATE TABLE public.interaction_logs (
  id UUID DEFAULT uuid_generate_v4() PRIMARY KEY,
  psid VARCHAR(255) REFERENCES public.leads(psid) ON DELETE CASCADE,
  message_type VARCHAR(50) CHECK (message_type IN ('user', 'ai_agent')),
  content TEXT NOT NULL,
  timestamp TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

Row Level Security (RLS) Configuration

We must expose the insert capability via Supabase's anonymous key for our serverless webhook, but strictly block `SELECT`, `UPDATE`, or `DELETE`.

```
-- Enable RLS
ALTER TABLE public.leads ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.interaction_logs ENABLE ROW LEVEL SECURITY;

-- Policies for 'leads'
CREATE POLICY "Allow Service Role Full Access on Leads"
  ON public.leads FOR ALL TO service_role USING (true);

CREATE POLICY "Allow Anon Insert on Leads"
  ON public.leads FOR INSERT TO anon WITH CHECK (true);

CREATE POLICY "Deny Anon Read on Leads"
  ON public.leads FOR SELECT TO anon USING (false);

-- Policies for 'interaction_logs'
CREATE POLICY "Allow Anon Insert on Logs"
  ON public.interaction_logs FOR INSERT TO anon WITH CHECK (true);

CREATE POLICY "Deny Anon Read on Logs"
  ON public.interaction_logs FOR SELECT TO anon USING (false);
```

4. Meta Messenger Bot & Lead Capture Integration

This module acts as the "Intelligent Routing" and "Lead Capture Engine". It receives webhooks from Meta, uses Gemini 1.5 Flash for Natural Language Understanding (intent detection), and pushes structured data to Supabase.

Node.js Webhook & AI Processor

```

const express = require('express');
const axios = require('axios');
const { GoogleGenerativeAI } = require('@google/generative-ai');
const { createClient } = require('@supabase/supabase-js');

const app = express();
app.use(express.json());

// Initialize Clients
const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);
const supabase = createClient(process.env.SUPABASE_URL, process.env.SUPABASE_ANON_KEY);
const PAGE_ACCESS_TOKEN = process.env.PAGE_ACCESS_TOKEN;

// Webhook Verification (Meta Requirement)
app.get('/webhook', (req, res) => {
  if (req.query['hub.verify_token'] === process.env.VERIFY_TOKEN) {
    res.status(200).send(req.query['hub.challenge']);
  } else {
    res.sendStatus(403);
  }
});

// Process Incoming Messages
app.post('/webhook', async (req, res) => {
  const body = req.body;
  if (body.object === 'page') {
    res.status(200).send('EVENT_RECEIVED'); // Acknowledge immediately to avoid timeouts

    for (const entry of body.entry) {
      const webhookEvent = entry.messaging[0];
      const senderPsid = webhookEvent.sender.id;

      if (webhookEvent.message && webhookEvent.message.text) {
        await processAgenticResponse(senderPsid, webhookEvent.message.text);
      }
    }
  } else {
    res.sendStatus(404);
  }
});

async function processAgenticResponse(psid, userMessage) {
  try {
    // 1. System Prompt for structured JSON extraction
    const model = genAI.getGenerativeModel({ model: "gemini-1.5-flash" });
    const prompt = `
    You are an elite AI sales agent for Nexus Automations PH.
    User message: "${userMessage}"
    Analyze the message and return a strictly formatted JSON object:
    {
      "intent": "inquiry|support|complaint|lead",
      "reply_message": "Your helpful, professional response.",
      "lead_score": 1-10
    }
    Do not include markdown formatting, just the raw JSON.
  `;
  }
};

```

```

const result = await model.generateContent(prompt);
const aiResponse = JSON.parse(result.response.text());

// 2. Send Response via Meta Graph API
await axios.post(
  `https://graph.facebook.com/v19.0/me/messages?access_token=${PAGE_ACCESS_TOKEN}`,
  {
    recipient: { id: psid },
    message: { text: aiResponse.reply_message }
  }
);

// 3. Upsert Lead and Log Interaction to Supabase
await supabase.from('leads').upsert({
  psid: psid,
  inferred_intent: aiResponse.intent,
  lead_score: aiResponse.lead_score,
  last_interaction: new Date().toISOString()
}, { onConflict: 'psid' });

await supabase.from('interaction_logs').insert([
  { psid: psid, message_type: 'user', content: userMessage },
  { psid: psid, message_type: 'ai_agent', content: aiResponse.reply_message }
]);

} catch (error) {
  console.error("Agentic Processing Error:", error);
  // Implement exponential backoff/retry here for production
}
}

app.listen(process.env.PORT || 3000, () => console.log('Agent Webhook Listening'));

```

5. Multi-Agent Content Creation & Auto-Posting Workflow

To maintain a 24/7 active social presence without manual intervention, we utilize **GitHub Actions** to trigger a daily cron job. A Python script calls Gemini to generate industry-relevant content, formats it, and publishes it via the Meta Graph API.

GitHub Actions Workflow (.github/workflows/auto-post.yml)

```
name: Autonomous Facebook Content Agent
on:
  schedule:
    - cron: '0 10 * * *' # Runs daily at 10:00 UTC
  workflow_dispatch: # Allows manual trigger

jobs:
  generate_and_post:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v3

      - name: Setup Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.10'

      - name: Install Dependencies
        run: pip install google-generativeai requests

      - name: Execute Agent Script
        env:
          GEMINI_API_KEY: ${ secrets.GEMINI_API_KEY }
          FB_PAGE_ACCESS_TOKEN: ${ secrets.FB_PAGE_ACCESS_TOKEN }
          FB_PAGE_ID: ${ secrets.FB_PAGE_ID }
        run: python scripts/agent_publisher.py
```


Python Agent Script (scripts/agent_publisher.py)

```
import os
import json
import requests
import google.generativeai as genai

def generate_post():
    genai.configure(api_key=os.environ["GEMINI_API_KEY"])
    model = genai.GenerativeModel('gemini-1.5-flash')

    prompt = (
        "Act as the Chief Marketing Officer for Nexus Automations PH. "
        "Write a highly engaging, professional Facebook post about the benefits of "
        "using autonomous AI agents in business. Include emojis and 3 relevant hashtags. "
        "Output ONLY the post content."
    )

    response = model.generate_content(prompt)
    return response.text.strip()

def publish_to_facebook(content):
    page_id = os.environ["FB_PAGE_ID"]
    access_token = os.environ["FB_PAGE_ACCESS_TOKEN"]
    url = f"https://graph.facebook.com/v19.0/{page_id}/feed"

    payload = {
        "message": content,
        "access_token": access_token
    }

    response = requests.post(url, data=payload)
    if response.status_code == 200:
        print(f"Successfully posted: {response.json()['id']}")
    else:
        print(f"Failed to post. Error: {response.text}")
        raise Exception("Facebook Graph API Post Failed")

if __name__ == "__main__":
    print("Initiating Content Generation Agent...")
    post_content = generate_post()
    print(f"Generated Content:
{post_content}
")
    print("Publishing to Meta Graph...")
    publish_to_facebook(post_content)
```

6. Production Maintenance, Self-Healing, and Scaling

Deploying AI agents into production requires rigid safeguards. To scale to 10,000+ daily interactions at zero cost, implement the following architectural SOPs:

- **Webhook Timeout Mitigation:** Meta requires a 200 OK response within 20 seconds. Always acknowledge the webhook `res.status(200)` synchronously *before* awaiting the LLM inference. Process the Gemini generation asynchronously.
- **Exponential Backoff:** If the Gemini API or Meta Graph API hits a rate limit (HTTP 429), implement a retry queue (e.g., using Redis, or Supabase as a queue table) multiplying the wait time by 2^n .
- **Model Fallback:** If Gemini 1.5 Flash experiences downtime, structure the AI processing function to gracefully fallback to a hardcoded rule-based response (e.g., "Our systems are currently prioritizing high load. An agent will contact you shortly.") to maintain user trust.
- **Secret Rotation:** Rotate the Meta Page Access Token and Gemini API Key every 90 days via GitHub Repository Secrets and Supabase Vault.

Scaling with n8n (Optional Alternative): While GitHub actions handle scheduled tasks perfectly, complex event-driven branching at scale (over 10,000 triggers) is best migrated to a self-hosted instance of n8n on a free Oracle Cloud ARM instance or an AWS EC2 micro tier.

Welcome to the future of automated business. Welcome to Nexus Automations PH.